

FIXPOINT SWARM-R

Phase 3: Autonomous Trading Instrument

Build Specification for Coding Agent

Historical replay, promotion workflow, sniper mode, multi-venue, risk dashboard, and TTCP-gated execution

- Input:** Phase 2 repository (16 crates, TUI, persistence, Binance adapter, TTCP engine, observability)
- Output:** Phase 3: a complete, deployable trading instrument capable of observation, backtesting, and progressive live execution
- Constraint:** Do not break Phase 1 or Phase 2 functionality. All 135+ existing tests must pass.
- Date:** 12 March 2026

Prime Directive

Phase 3 completes the journey from engine to instrument. After Phase 3, a solo operator can: record live market data, replay it for backtesting, validate a configuration in paper-mode, promote it through a formal gate, and execute in live-mode with TTCP-gated risk control — all from one TUI. No new crates. All additions go into existing crates.

Contents

1	Phase 3 Scope	2
1.1	What Phase 3 Adds	2
1.2	What Phase 3 Does NOT Add	2
1.3	Module Placement	2
2	Historical Data Recorder	2
2.1	Recording Format	3
2.2	Recording Activation	3
2.3	Historical Replay	3
3	Backtest Report	4
4	Promotion Workflow	5
4.1	Workflow Steps	5
4.2	Promotion Gate Checks	6
5	Sniper Mode	6
5.1	Concept	6
5.2	Execution Gating	6
5.3	Position Scaling	7
5.4	Progressive Risk Escalation	7
5.5	TUI Sniper Indicator	7
6	Risk Dashboard	8
6.1	Risk Metrics	8
6.2	Risk Limits	8
7	Multi-Venue: Kraken Adapter	9
7.1	Implementation	9
7.2	Cross-Venue Candidate Discovery	9
7.3	Venue Configuration	9
8	Updated TUI Layout	10
9	Updated CLI Surface	10
10	Implementation Order	11
11	Definition of Done (Phase 3)	12
12	Phase 4 Preview (Non-Normative)	12

1 Phase 3 Scope

1.1 What Phase 3 Adds

1. **Historical Data Recorder & Replay:** Record live Binance order books to disk in a compact, replayable format. Replay recorded data through the macro-cycle as if it were live. This enables backtesting on real market data without network access.
2. **Promotion Workflow:** Formal, gate-controlled pipeline from paper-validated configuration to live execution. The Promotion FSM (already implemented in Phase 1) is now wired to an operational workflow with CLI commands and TUI status.
3. **Sniper Mode:** A new execution discipline where the system observes passively and only executes when TTCP crystallizes. Progressive risk escalation: position size scales with crystal confidence (convergence distance).
4. **Risk Dashboard:** Extended TUI panel showing real-time exposure, drawdown, position inventory, and risk budget consumption. The operator sees at a glance whether the system is within safe parameters.
5. **Multi-Venue:** Second venue adapter (Kraken Spot) with the same trait interface. Cross-venue candidate discovery for triangular arbitrage.
6. **Backtesting Report:** After a historical replay run, emit a structured report (JSON + human-readable summary) with P&L curve, drawdown, win rate, Sharpe ratio, and TTCP crystal statistics.

1.2 What Phase 3 Does NOT Add

- No web GUI, no REST API, no mobile interface.
- No ML/AI prediction components.
- No derivatives, margin trading, or futures.
- No multi-operator support.
- No new crates (16 crate budget maintained).

Crate Freeze

Phase 3 adds zero new crates. All new functionality is implemented as modules within existing crates. The workspace remains at exactly 16 crates.

1.3 Module Placement

Feature	Crate	Module
Data recorder	fsr-runtime	src/recorder.rs
Historical replay	fsr-runtime	src/replay_historical.rs
Promotion workflow	fsr-calibration	src/promotion_workflow.rs
Sniper mode	fsr-runtime	src/sniper.rs
Risk dashboard	fsr-tui	src/panels/risk.rs
Kraken adapter	fsr-runtime	src/kraken.rs
Backtest report	fsr-runtime	src/backtest_report.rs

2 Historical Data Recorder

2.1 Recording Format

Each tick of the macro-cycle in live-observation mode writes a **MarketFrame**:

Listing 1: MarketFrame struct

```
pub struct MarketFrame {
    pub timestamp_us: u64,          // wall-clock microseconds
    pub tick: u64,                  // macro-cycle tick
    pub venue: VenueId,
    pub pairs: Vec<PairSnapshot>,
}

pub struct PairSnapshot {
    pub pair: TradingPair,
    pub bids: Vec<(Q32, Q32)>,     // (price, quantity), top N
    pub asks: Vec<(Q32, Q32)>,
    pub mid_price: Q32,
    pub spread_bps: Q32,
}
```

Frames are written as length-prefixed bincode to `data/recordings/{session}.rec`. Each file is a contiguous, seekable stream. Files rotate every 100 MB.

2.2 Recording Activation

Listing 2: CLI for recording

```
# Observe Binance live and record order books to disk
fsr run-live --venue binance --record

# Output: data/recordings/{run_id}_000.rec, _001.rec, ...
```

Recording runs in a background task alongside the macro-cycle. The recorder receives **MarketFrames** from the venue adapter's order book cache. No additional API calls — it captures the same data the macro-cycle already sees.

2.3 Historical Replay

Listing 3: CLI for historical replay

```
# Replay recorded data through the macro-cycle
fsr replay-historical --recording data/recordings/{session}*.rec \
    --config config/conservative.yaml \
    --output data/backtest/{report_id}/
```

The replay engine:

1. Opens the `.rec` files in order.
2. Creates a **HistoricalFeed** that implements **PaperFeed**, yielding one **MarketFrame** per macro-cycle tick.
3. Runs the full macro-cycle (gates, CSP, hedge, TTCP — everything) against the recorded data.
4. Writes all output to `data/backtest/{report_id}/`: chain files, snapshots, logs, and the backtest report.

Listing 4: HistoricalFeed

```
pub struct HistoricalFeed {
    frames: Vec<MarketFrame>, // loaded from .rec files
    cursor: usize,
}

impl PaperFeed for HistoricalFeed {
    fn next_tick(&mut self) -> Option<MarketTick> {
        let frame = self.frames.get(self.cursor)?;
        self.cursor += 1;
        Some(frame.into()) // convert MarketFrame -> MarketTick
    }
    fn reset(&mut self, _seed: u64) { self.cursor = 0; }
    fn is_deterministic(&self) -> bool { true }
}
```

Replay Determinism

Historical replay must be bit-identical across runs: same recording + same config = same chain digests. The `HistoricalFeed` feeds frames in recorded order with no randomness. This is not synthetic data — it is recorded reality replayed deterministically.

3 Backtest Report

After a historical replay completes, the engine emits a structured report:

Listing 5: BacktestReport struct

```
pub struct BacktestReport {
    pub run_id: String,
    pub config_hash: Hash256,
    pub recording_files: Vec<String>,
    pub total_ticks: u64,
    pub duration_covered: Duration, // wall-clock span of recorded data

    // P&L
    pub net_pnl_bps: Q32,
    pub gross_pnl_bps: Q32,
    pub total_fees_bps: Q32,
    pub max_drawdown_bps: Q32,
    pub sharpe_ratio: Q32, // annualized, computed from tick returns

    // Execution
    pub total_trades: u64,
    pub settled_trades: u64,
    pub aborted_trades: u64,
    pub win_rate: Q32, // settled with positive PnL / total settled
    pub avg_trade_pnl_bps: Q32,

    // Regime
    pub ticks_alpha: u64,
    pub ticks_beta: u64,
    pub ticks_gamma: u64,

    // TTCP
    pub ttcp_crystals_emitted: u64,
    pub ttcp_avg_delta1: Q32,
```

```

pub ttcp_avg_delta2: Q32,
pub ttcp_avg_delta3: Q32,

// Integrity
pub invariant_violations: u64, // should be 0
pub safe_hold_entries: u64,
pub chain_digest_final: Hash256,
}

```

Output:

- `data/backtest/{report_id}/report.json`: machine-readable.
- `data/backtest/{report_id}/summary.txt`: human-readable one-page summary printed to terminal after replay.

Listing 6: Sample summary.txt output

```

=== BACKTEST REPORT: run_2026-03-12_14h ===
Config:    balanced.yaml (hash: a4f2c8...)
Recording: 2026-03-10 to 2026-03-12 (48h, 172800 ticks)
---
Net P&L:      +23.7 bp
Max Drawdown: -8.4 bp
Sharpe (ann.): 1.84
Win Rate:     62.1% (18/29 settled)
Aborted:      11
---
Regime:  Alpha 78% | Beta 19% | Gamma 3%
TTCP:     4 crystals | avg d1=0.032 d2=0.071 d3=0.109
Integrity: 0 violations | 0 safe-holds
Chain:    a8b3f1... (verified)
===

```

4 Promotion Workflow

Phase 1 implemented the Promotion FSM (Candidate → PaperValidated → PromotionPending → LiveEligible → LiveActive). Phase 3 wires this FSM to an operational workflow.

4.1 Workflow Steps

1. **Candidate**: Operator creates a new config variant (e.g., adjusts gate thresholds). Saves as YAML.
2. **Paper Validation**: Operator runs:

```
fsr promote --config candidate.yaml --recording data/recordings/latest_*.rec
```

This replays the candidate config against recorded data and generates a BacktestReport. If $\text{net PnL} \geq 0$ and no invariant violations: state → PaperValidated.

3. **Promotion Proposal**: If paper passes, the system emits a `PromotionProposal` artifact containing: candidate config hash, backtest report hash, P&L summary, TTCP crystal count. State → PromotionPending.
4. **Operator Approval**: The operator reviews the proposal in the TUI or via:

```
fsr promote --approve {proposal_id}
```

If `live_promotion_requires_human_approval: true` (default), this step is mandatory.
 State $\rightarrow$ LiveEligible.

5. **Live Activation:** Operator starts live mode with the promoted config:

```
fsr run-live --config promoted.yaml --venue binance [--sniper]
```

State $\rightarrow$ LiveActive. The system enters the 100-tick observation-only warm-up before any execution.

6. **Demotion:** If live P&L drops below a configured threshold or an invariant violation occurs, automatic demotion back to PaperValidated. Evidence-logged.

4.2 Promotion Gate Checks

The `fsr promote` command enforces these gates (from the Phase 1 specification):

1. **Replay Gate:** Backtest is deterministically reproducible.
2. **Benchmark Gate:** Net PnL ≥ 0 after frictions.
3. **Stability Gate:** Win rate $\geq \text{min_win_rate}$ (configurable, default 55%).
4. **Invariant Gate:** Zero invariant violations during backtest.
5. **TTCP Gate:** At least 1 TTCP crystal emitted during backtest (the market showed topological stability at least once).

Promotion Rule

No configuration may reach LiveActive without passing all 5 gates. The PromotionProposal artifact is committed to the commitment chain and is part of the permanent audit trail.

5 Sniper Mode

Sniper mode is the operational realization of the “observe first, act selectively” discipline.

5.1 Concept

In sniper mode, the system runs the full macro-cycle (observation, resonance, gating, TTCP) but **suppresses all execution** unless a TTCP crystal has been emitted within the last W_{sniper} ticks. The crystal acts as the “authorization to engage.”

5.2 Execution Gating

Listing 7: Sniper gate logic

```
pub fn sniper_gate(
  ttcp: &TtcpState,
  tick: u64,
  config: &SniperConfig,
) -> SniperDecision {
  let ticks_since_crystal = tick.saturating_sub(
    ttcp.last_crystal_tick.unwrap_or(0)
  );

  if ticks_since_crystal > config.crystal_validity_window {
    return SniperDecision::Observe; // no execution allowed
  }

  // Crystal is fresh -- scale position by convergence confidence
```

```

    let confidence = compute_confidence(ttcp, config);
    SniperDecision::Execute { scale_factor: confidence }
}

fn compute_confidence(ttcp: &TtcpState, config: &SniperConfig) -> Q32 {
    // Confidence = 1 - (max_delta / threshold)
    // Ranges from 0 (barely crystallized) to 1 (deeply converged)
    let max_delta = ttcp.delta1.max(ttcp.delta2).max(ttcp.delta3);
    let raw = Q32::ONE - max_delta / config.crystallization_threshold;
    raw.clamp(config.min_scale, config.max_scale)
}

```

5.3 Position Scaling

In sniper mode, every order's size is multiplied by the `scale_factor`:

- `scale_factor` = 0.0: no execution (Observe mode).
- `scale_factor` = 0.1: 10% of normal position size (barely crystallized, low confidence).
- `scale_factor` = 1.0: full position size (deeply converged, high confidence).

The `min_scale` and `max_scale` configuration parameters bound this range. Default: `min_scale` = 0.05 (never less than 5% position), `max_scale` = 0.50 (never more than 50% in sniper mode — full size requires explicit non-sniper mode).

5.4 Progressive Risk Escalation

The operator controls risk escalation through config parameters, not through code changes:

Listing 8: Sniper config section

```

sniper:
  enabled: true
  crystal_validity_window: 50    # ticks a crystal remains valid
  min_scale: 500                 # Q32: 0.05 = 5% position
  max_scale: 5000                # Q32: 0.50 = 50% position
  require_consecutive_crystals: 1 # must see N crystals before first trade
  cooldown_after_loss: 100       # ticks of observation-only after a loss

```

The escalation path:

1. Start with `max_scale: 500` (5%). Observe for days/weeks.
2. If results are consistently positive: increase to `max_scale: 2000` (20%).
3. If still positive over weeks: increase to `max_scale: 5000` (50%).
4. Never increase to 100% in sniper mode. Full position = deliberate switch to non-sniper mode.

Sniper Philosophy

The system does not decide when to escalate. The **operator** decides, based on observed results. The system provides: (a) clear metrics on every decision (backtest reports, crystal counts, P&L history), (b) hard enforcement of whatever limits the operator sets, (c) automatic cooldown after losses. The escalation is progressive, manual, and evidence-based.

5.5 TUI Sniper Indicator

Panel 1 (Regime & Gates) is extended with a sniper status line:

Listing 9: TUI sniper display

```

Sniper: ARMED  crystal age: 12 ticks  scale: 0.31  [|||....]
Sniper: OBSERVE no crystal (247 ticks) scale: 0.00  [.....]

```

The bar [|||....] visualizes the scale factor. “ARMED” means a fresh crystal authorizes execution. “OBSERVE” means passive observation only.

6 Risk Dashboard

Panel 4 in the TUI (previously “P&L & Execution”) is expanded into a full risk panel:

Listing 10: Risk panel layout

```

[4] Risk & P&L
Net PnL:   +23.7 bp      Gross: +28.5 bp   Fees: -4.8 bp
Drawdown:  -3.1 bp      Max DD: -8.4 bp   DD limit: -30.0 bp
Exposure:  127.4 units  Leverage: 0.42x   Cap: 5.00x
Trades:    29 (18W/11L) Win: 62.1%      Avg: +0.82 bp
Risk used:  14.2%      Budget: 10.00%   [||.....]
Inventory:  BTC +0.003  ETH -0.012    USDT +47.21

```

6.1 Risk Metrics

Metric	Computation	Source
Net PnL	Sum of settled trade PnLs	Commitment chain receipts
Max Drawdown	Deepest peak-to-trough in cumulative PnL	Running computation
Exposure	Sum of absolute position values	Inventory snapshot
Leverage	Exposure / total capital	Config + inventory
Risk Used	Current drawdown / risk_budget	Config + PnL
Inventory	Per-asset net position	Filled orders

6.2 Risk Limits

The existing governance system (INV-10: hedge leverage cap, INV-14: resource exhaustion) is extended with:

Listing 11: Risk limit config

```

risk:
  max_drawdown_bps: 3000      # -30 bp -> forced safe-hold
  max_leverage: 5.0
  max_position_per_pair_bps: 10000 # max 100 bp exposure per pair
  daily_loss_limit_bps: 1500    # -15 bp in 24h -> pause execution
  risk_budget_fraction: 0.10   # 10% of capital at risk

```

If `daily_loss_limit_bps` is breached, the system enters observation-only mode for the remainder of the 24h window. This is automatic, not operator-discretionary.

7 Multi-Venue: Kraken Adapter

7.1 Implementation

Same trait (`VenueAdapter`) as `Binance`. Feature-gated behind `-features live-data`.

Listing 12: Kraken adapter skeleton

```
pub struct KrakenAdapter {
    ws_handle: Option<JoinHandle<()>>,
    orderbook_cache: Arc<Mutex<HashMap<TradingPair, OrderBook>>>,
    api_key: Option<String>,
}

impl VenueAdapter for KrakenAdapter {
    fn venue_id(&self) -> VenueId { VenueId::Kraken }
    // Same interface as BinanceAdapter
}
```

7.2 Cross-Venue Candidate Discovery

With two venues, the candidate discovery in `fsr-candidates` can find cross-venue cycles: e.g., buy BTC on Binance (lower ask), sell on Kraken (higher bid). The trumpet L1 (triangular arb) now includes cross-venue legs.

Listing 13: Cross-venue candidate

```
pub struct CandidateLeg {
    pub venue: VenueId,
    pub pair: TradingPair,
    pub side: Side, // Buy or Sell
    pub price: Q32,
    pub available_qty: Q32,
}

// A cross-venue triangular cycle:
// Leg 1: Buy ETH/USDT on Binance
// Leg 2: Sell ETH/BTC on Kraken
// Leg 3: Sell BTC/USDT on Binance
```

7.3 Venue Configuration

Listing 14: Multi-venue config

```
venues:
  binance:
    enabled: true
    pairs: ["BTCUSDT", "ETHUSDT", "ETHBTC"]
    # ... (same as Phase 2)
  kraken:
    enabled: true
    pairs: ["XBTUSDT", "ETHUSDT", "ETHXBT"]
    ws_endpoint: "wss://ws.kraken.com/v2"
    api_key_env: "KRAKEN_API_KEY"
    api_secret_env: "KRAKEN_API_SECRET"
    execution_enabled: false
```

```

cross_venue:
  enabled: true
  pair_mapping:          # normalize pair names across venues
    BTCUSDT: ["BTCUSDT", "XBTUSDT"]
    ETHUSDT: ["ETHUSDT", "ETHUSDT"]
    ETHBTC:  ["ETHBTC", "ETHXBT"]
  max_cross_venue_legs: 3 # max legs in cross-venue cycle

```

Cross-Venue Safety

Cross-venue execution is non-atomic by nature (different exchanges cannot coordinate). The CSP must account for execution risk: if leg 1 fills but leg 2 fails, the system has unhedged exposure. In sniper mode, cross-venue cycles are only attempted when `scale_factor` ≥ 0.3 (moderate confidence). Below that, only single-venue cycles are allowed.

8 Updated TUI Layout

Listing 15: Phase 3 TUI layout

```

+--[ FIXPOINT SWARM-R v3.1.0 ]--[ tick: 4821 ]--[ live|sniper ]-+
|
|  [1] Regime & Sniper          [2] Resonance Metrics          |
|  Regime: Alpha                kappa: 0.7234    SI: 1247 bp    |
|  Gate: OPEN (G=0.712)         psi/rho/omega: .81/.74/.69    |
|  Sniper: ARMED scale=0.31      Leak: 0.0312     MCI: 0.891    |
|  Hedge: Safe   CSP: Idle                                     |
|-----+-----|
|  [3] Candidates & Venues      [4] Risk & P&L                  |
|  BIN L1: 8  L2: 2             Net: +23.7 bp   DD: -3.1/-30 bp |
|  KRK L1: 5  L2: 1             Exp: 127.4   Lev: 0.42x/5.00x   |
|  X-venue: 3                   Trades: 29 (62.1%) Risk: 14.2%  |
|  Press top-3: SI=1891 bp      Inventory: BTC+.003 ETH-.012   |
|-----+-----|
|  [5] Event Log                |
|  04:12:03.441 [TTCP] Crystal #4 emitted (d1=.031 d2=.068)    |
|  04:12:03.442 [SNIPER] ARMED -> Execute (scale=0.31)         |
|  04:12:03.501 [CSP] Settled: +1.2 bp (BIN tri-leg)           |
|-----+-----|
|  [6] TTCP & Crystals          |
|  Cascade: 3/4 d1=0.031 d2=0.068 d3=0.104                    |
|  Crystals: 4 total Last: 12 ticks ago                        |
|  Crystal validity: 38 ticks remaining                        |
|-----+-----|
|  [q]uit [p]ause [r]esume [c]onfig [s]niper-toggle [+/-]speed |
|-----+-----|

```

New keybinding: `s` toggles sniper mode on/off (requires confirmation prompt in TUI).

9 Updated CLI Surface

Listing 16: Phase 3 CLI commands

```
# Existing (unchanged)
fsr run-paper --config <path> [--no-tui] [--resume <run_id>]
fsr run-live --venue <name> --config <path> [--no-tui] [--record]
fsr replay --checkpoint <path>
fsr audit --chain <path>
fsr validate-config --config <path>
fsr status
fsr runs [--last <n>]

# New in Phase 3
fsr replay-historical --recording <path> --config <path> \
    --output <dir>
fsr promote --config <path> --recording <path>
fsr promote --approve <proposal_id>
fsr promote --status
fsr backtest-report --run <run_id>
```

10 Implementation Order

Step	Component	Done when
3.1	Data recorder	fsr run-live -record writes .rec files. Files load back correctly.
3.2	Historical replay	fsr replay-historical runs recorded data through macro-cycle. Chain digests are deterministic across repeated runs.
3.3	Backtest report	After historical replay, report.json and summary.txt are emitted with correct P&L, drawdown, Sharpe, win rate.
3.4	Promotion workflow	fsr promote runs backtest, checks 5 gates, emits proposal. fsr promote -approve transitions FSM. Event in commitment chain.
3.5	Risk dashboard	TUI panel 4 shows exposure, leverage, drawdown, risk budget, inventory. All limits enforced. Daily loss limit triggers auto-pause.
3.6	Sniper mode	-sniper flag. TTCP-gated execution. Scale factor computed from convergence. TUI shows sniper status. Cooldown after loss.
3.7	Kraken adapter	-venue kraken works. Cross-venue candidates discovered. Pair name normalization.
3.8	Integration test	Full pipeline: record live data → replay → promote → run-live sniper mode. All Phase 1+2 tests pass.

Step Closure Rule

Each step must compile, pass tests, and not break prior steps. Strict sequential execution.

11 Definition of Done (Phase 3)

Phase 3 is complete when:

1. All Phase 1 and Phase 2 tests pass unchanged.
2. `cargo clippy -workspace - -D warnings` passes.
3. `fsr run-live -venue binance -record` records order books to `.rec` files.
4. `fsr replay-historical -recording {file} -config balanced.yaml` replays deterministically and emits a backtest report.
5. `fsr promote -config candidate.yaml -recording {file}` runs the 5-gate promotion workflow.
6. `fsr promote -approve {id}` transitions the Promotion FSM and logs to commitment chain.
7. `fsr run-live -venue binance -sniper` runs in sniper mode with TTCP-gated execution.
8. Sniper mode respects scale factor, cooldown, and daily loss limit.
9. TUI shows sniper status, risk dashboard, and cross-venue candidates.
10. `fsr run-live -venue binance -venue kraken` connects to both venues.
11. Cross-venue candidates appear in trumpet L1 when price discrepancies exist.
12. No network access required for `fsr run-paper` or `fsr replay-historical`.

12 Phase 4 Preview (Non-Normative)

- Full persistent homology (Ripser bindings) for TTCP.
- Portfolio-level risk management (multiple concurrent strategies).
- Strategy rotation based on regime detection.
- Automated promotion: if a config passes all gates N times consecutively, promote without manual approval.
- Mobile notification via Telegram/Signal when Crystal emits or safe-hold triggers.

End of Phase 3 Specification